The Relay: cited answers from three papers under real rate limits

Code: github.com/Dishan18/Vaani (also in this folder). Raw data: results.csv (one row per request), data/probe/probe_raw.jsonl (every probe call with headers and 429 bodies), data/results/detail.jsonl (per-step spans), data/experiments/ (tuning runs). Conversation with the assistant: conversation/ folder.

1 What I built and why

A Python asyncio pipeline: **understand** (qwen3.8-27b picks the papers and rewrites the query) then **retrieve** (BM25 search tool over section-aware pieces; loops once with a second query if evidence is weak) then **decide** (refuse with no model call if the evidence is too weak) then **draft** (gpt-oss-120b, every claim cited as [S#]) then **check** (a deterministic citation gate always; a gpt-oss-20b verifier only when there is headroom) then **send**. Each Groq model has its own buckets, so spreading steps across three models triples capacity.

The decision that shaped it most: never make a person wait for budget that will not come. One answer costs about 1,700 tokens and each model allows 8,000 tokens a minute, so the three models write about 10 full answers a minute. Above that something must give. Before every model call a client-side mirror of Groq's buckets (synced from headers and 429 bodies) predicts the wait, counting the queue ahead. If the drafting model is already backed up, the request goes lean (draft only, 4 pieces, about half the tokens). If the preferred model is not free within 2 s, it takes another model that is. If nothing frees within 3 s, the person gets the cited passages with no model call; only an internal failure gives "busy, retry in N s".

2 The real service and my stand-in

I probed Groq with 119 requests (about 98K tokens) in five phases: headers, an RPM wall, a TPM wall, concurrency bursts and paced latency (docs/probe_plan.md).

Limit	Value you found	How you found it	Confidence
Requests per minute	30 per model	30 tiny calls to gpt-oss-20b in 4 s; call 31 got 429 "requests per minute (RPM): Limit 30, Used 30"	high
Tokens per minute	8,000 per model; qwen also has 7,000 input tokens/min (ITPM)	`x-ratelimit-limit-tokens: 8000` on every reply; 429 bodies "TPM: Limit 8000" and, for qwen only, "ITPM: Limit 7000" (no header shows it)	high
Requests per day	1,000 per model, refilled continuously at 1 request per 86.4 s	`x-ratelimit-limit-requests: 1000`; `reset-requests / requests used` = 86.40 s on all 119 probe calls. Not hit on purpose	high (value), medium (behaviour at zero)
Tokens per day	200,000 per model (docs)	Groq docs table only; no header reports it, and no run (probe, Table 3, real loads) ever got a daily-limit 429	low
Requests in flight at once	no limit seen up to 8	bursts of 1/4/8/16 to qwen: 8 at once all OK with no slowdown; at 16 the ITPM limit hit first, so higher is unknown	low
What it returns when a limit is hit	HTTP 429, JSON `error.type` requests or tokens, `code` rate_limit_exceeded, message with Limit / Used / Requested; `retry-after` in seconds sized to this request; an RPM 429 has no token headers	raw 429 responses in data/probe/probe_raw.jsonl	high
Time until it recovers	about 2 s for a small request; `retry-after` said 2 s (RPM) and 21 s (TPM, for a 4,319-token request)	first poll 2 s after each 429 succeeded; buckets refill continuously (RPM 0.5/s, TPM 133 tokens/s)	high

Three findings changed the design. **Limits refill continuously**: `reset-requests` divided by requests used was exactly 86.40 s on every call, so the day never "resets", it trickles back. **Groq charges at admission for prompt plus the full max_tokens** (13,208 characters with max_tokens 16 was "Requested 4319"), so my limiter reserves the same way and refunds after; before that fix 16 of 72 drafts hit a surprise 429. **gpt-oss can return HTTP 200 with no text** when hidden reasoning uses up max_tokens; the router on gpt-oss-20b did this 5.2% of the time (median 569 ms), so the router moved to qwen (220 ms, no empties).

Stand-in (standin/server.py, OpenAI-compatible streaming): the same buckets, headers, 429 bodies and size-dependent retry-after; first token drawn from a lognormal fitted to each model's probe p50/p95 (120b 695/936 ms, 20b 659/909, qwen 352/489); output streamed in chunks at the measured 633/1,522/486 tokens/s; switches for 503s, 8 s stalls, halved limits and an empty day. **Where it is least like Groq:** its answers are synthetic (it quotes the first source), so quality is graded only on real runs; its slowdown under load (4% per request in flight above 4) is an assumption, because Groq showed none at 16 concurrent; and it starts with full buckets, while the real account was already partly spent.

3 Results

Real runs: real_steady, real_burst and all of Table 3. Everything else used the stand-in. Percentiles are p50/p95/p99 by linear interpolation over the "Requests sent" in each row (n is small, so p99 is close to the maximum). First token counts every reply that released text; total time includes people who gave up. Rate-limited includes passages-only replies. Peak queue counts requests beyond 12 running pipelines.

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
real_steady	real	14	173	4.8	1654 / 2505 / 2521	1654 / 2505 / 2521
real_burst	real	10	3	206.0	1983 / 2488 / 2644	1983 / 2488 / 2644
steady	stand-in	17	103	9.9	1616 / 2452 / 3243	1616 / 2452 / 3244
burst	stand-in	42	58	43.1	2 / 2665 / 2889	3 / 2665 / 2889
overload	stand-in	64	121	31.9	632 / 2645 / 3339	632 / 2645 / 3339
overload_naive_retry	stand-in	64	121	31.8	1468 / 1917 / 2795	1468 / 1917 / 2795
rpm_halved	stand-in	17	102	10.0	552 / 2087 / 2386	552 / 2087 / 2386
daily_exhausted	stand-in	10	57	10.5	1 / 13 / 18	1 / 13 / 18
chaos	stand-in	17	102	10.0	1933 / 3497 / 3752	1956 / 4066 / 4865

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
real_steady	100.0	0.0	0.0	0.00	0	2082	$0 free tier ($0.40 at list price)
real_burst	90.0	10.0	0.0	0.10	0	2019	$0 free tier ($0.39 at list price)
steady	100.0	0.0	0.0	0.00	0	2739	$0 free tier ($0.49 at list price)
burst	31.0	69.0	0.0	0.07	0	2012	$0 free tier ($0.36 at list price)
overload	82.8	17.2	0.0	0.00	0	1303	$0 free tier ($0.25 at list price)
overload_naive_retry	28.1	71.9	0.0	2.47	0	1958	$0 free tier ($0.36 at list price)
rpm_halved	100.0	0.0	0.0	0.00	0	1378	$0 free tier ($0.27 at list price)
daily_exhausted	0.0	100.0	0.0	0.20	0	0	$0 free tier ($0.00 at list price)
chaos	94.1	0.0	5.9	0.06	0	2409	$0 free tier ($0.44 at list price)

Answer quality on 18 questions (14 answerable, 4 not, three of them traps): the chosen setting answered 14/14 correctly with citations and refused 4/4 correctly, at 1,679 tokens per answer. Under real steady load 11 of 14 were correct, 2 correctly refused, and 1 correct answer was wrongly rejected by the LLM verifier (q11). **Numbers I could not have guessed:** the daily refill of one request per 86.4 s; qwen's hidden 7,000 input-tokens-per-minute limit, visible only in 429 bodies; and that a naive client under the same overload sent 240 calls and got 204 429s, against 70 calls and one 429 for mine.

4 Where the time goes

The slowest request in the chaos run (stand-in; 5% of calls stall for 8 s) is chaos-000541-0007, 3,816 ms (scripts/explain_request.py):

Step	Model	Start to end (ms)	Kind
understand	qwen3.8-27b	0 to 337	working
retrieve	local BM25	338 to 338	working
draft	qwen3.8-27b (120b was busy)	339 to 791	working
verify	gpt-oss-20b	791 to 3,816	failed: no first token in 3 s, cut and skipped

Waiting 0 ms, working 790 ms, retrying (failed attempt) 3,025 ms. All of it came **before the first token**, because an answer is released only after its checks finish. I know this from the spans: the pipeline times every model call and every budget wait and saves them per request in data/results/detail.jsonl; the first-token time is in results.csv. Two fixes came from requests like this one. The first-token timeout went from 6 s to 3 s: the slowest healthy real first token in 345 calls was 2,240 ms, and with 20% stalls a 3 s cut gave 94.1% success against 76.5%. And best-effort steps are no longer retried: before that, the slowest chaos request took 8,073 ms, 75% of it retrying.

5 When a limit is hit

Per-minute limit: delayed by at most 2 s are people whose preferred model is about to free up. After that they get a weaker answer: another model now, or lean mode when the drafting model is backed up (lean: 12/14 correct against 14/14, at 874 tokens instead of 1,679). If nothing frees within 3 s they get the cited passages with no model call (status rate_limited), not a long wait. Nobody is dropped silently. With the per-minute limits halved and the client not told (rpm_halved), it read the new limit from the first reply's `x-ratelimit-limit-tokens` header, went lean (1,378 tokens per answer against 2,739 in steady) and succeeded 100% with no 429s. **Per-day limit:** below 15% of the day left on the drafting model, everyone goes lean, which stretches the rest about twice as far. At zero (daily_exhausted) every person gets cited passages in under 20 ms; the client learned the empty day from the first 429 body. Because the day refills one request every 86.4 s, "gone" is never permanent, and the reply says roughly when a full answer will be possible again.

Retrieval: quality versus cost

Setting (piece size, pieces fetched, re-rank yes or no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
medium(piece=1200c,k=5,rerank=no)	18	72.2	0.0	22.2	1781	7512 / 12048	7512 / 12048
small(piece=600c,k=8,rerank=no)	18	77.8	0.0	22.2	1679	8685 / 11324	8685 / 11324
large(piece=2400c,k=3,rerank=no)	18	72.2	0.0	22.2	1996	8184 / 13963	8184 / 13963
medium_rerank(piece=1200c,k=4,rerank=yes)	18	72.2	5.6	22.2	2692	11868 / 15151	11868 / 15151
small_lean(piece=600c,k=4,rerank=no)	18	66.7	11.1	22.2	874	729 / 7886	729 / 7886

All settings drafted on gpt-oss-120b with the verifier off, so only retrieval differs (16 of the first 72 drafts fell back to qwen after the 429 issue fixed above). The differences are one question each, which is within noise, so cost decides: **small pieces (600 characters, 8 fetched, no re-rank)** were never worse and used the fewest tokens. Re-ranking cost 51% more tokens for nothing. Table 3's latencies are mostly time spent waiting for 120b's bucket (the runs were sequential and pinned to one model); the drafting call itself has a median of 678 ms over 92 real calls. **When the limit is close**, retrieval shrinks: 4 pieces instead of 8, no router, no re-rank, which halves input tokens (the small_lean row shows the cost in quality).

How this library will mislead a naive retriever

(Raw question, plain BM25, references included; scripts/retriever_traps.py.) 1. **Same term, wrong paper.** "What word error rate does H-Net achieve on LibriSpeech?" The top hit is Moshi 5.7, where LibriSpeech WER is 4.7%, so a naive pipeline answers 4.7% for H-Net. Mine refused it in all five settings, including lean mode with no router: the drafting prompt says to answer only from the sources, and the sources were plainly about Moshi. 2. **Vocabulary mismatch.** "What is the frame rate of the Mimi codec?" The passage that says "12.5Hz" ranks 18th, out of reach of 5 or 8 pieces, because the paper never says "frame rate" there. My q06 also asks for the bitrate, and "1.1kbps" sits in the same passage, so it was answered in every setting; asked alone, the question is the trap. The router's rewritten query is my defence, but my test set does not prove it works. 3. **Tables and numbers break in extraction.** Attention's Table 2 lands under "5.4 Regularization", and its training cost 2.3 x 10^19 FLOPs is extracted as "2.3 · 1019", so "What did the big model cost in FLOPs?" can be answered off by 18 orders of magnitude. The drafting prompt warns about flattened exponents; the citation still points at the wrong section.

6 Retries

At most one retry per step, never on the bucket that just returned 429 (it switches model), never after a daily-limit 429, and retries capped at 10% of all attempts so they cannot multiply load. Best-effort steps (verifier, re-rank) are never retried; a call with no text counts as failed. **Retrying made it worse** in overload_naive_retry: a client with no limiter that retries the same model up to 3 times. Same arrivals as overload: 240 calls and 204 429s, 28.1% succeeded, against 70 calls, one 429 and 82.8% for mine. Each retry hit a bucket that was still empty and used up the person's time.

7 What the person sees

At 3pm with the daily budget gone they get, in under 20 ms: "I can't write a full answer right now (our model capacity is used up), but these are the passages from the papers that match your question:" followed by up to three passages, each with paper, section and page and a 300-character quote, then "A full answer should be possible again in about 1 min." The estimate comes from the bucket's refill rate. They get evidence, not a guess and not a spinner.

8 What I'm unsure about, what I left out, what I learned

Least certain assumption: that limits are per model and independent. Each 429 names a model and three simultaneous walls did not interfere, but if Groq pooled them per organisation, capacity would be a third of what I plan for and the ladder would kick in three times sooner. Also untested: real daily-limit behaviour at zero, and in-flight limits above 8. **Deliberately not built:** embeddings or vector search (another rate-limited dependency, and BM25 with a router already got 14/14); streaming tokens before the check (holding a 250-token draft costs well under a second on Groq, and I never have to retract an answer). **New to me:** a rate limit is a bucket that refills, not a window that resets. Groq's "1,000 requests a day" is a bucket of 1,000 that gains one request every 86.4 s. That changes the engineering: a 429 is not "come back tomorrow" or even "come back in a minute", it is "come back when enough has dripped back in for a request your size". That is why a big request is told to wait 21 s while a tiny one gets through 2 s later, and why the right response to a limit is to make the request smaller (lean mode) or send it to another bucket (another model), not to retry the same request.

Your feedback on Vaani

What stood out: real volumes in the case studies (69,990 calls placed, 305 hours of automated talk time) and a percentile latency claim ("~450ms p90") instead of an average, which is the right instinct and exactly what this assignment trains. What confused me: what a "voice AI harness" is, and how Teacher AI, Memories and Call screening fit together, and whether the page is for developers or for enterprise buyers. One thing I would change: the headline numbers are animated counters that read 0 until scrolled into view, so link previews, crawlers and AI summaries of the site see "0M+" and "<0ms". I hit this myself when I fetched the page. I would put the real values in the HTML, and say next to the latency claim what is measured (first audio byte?) and where.

How I worked with the assistant

I used Claude Code (Claude Opus) for the whole build. The real, unedited conversation is in the conversation/ folder; the only change is that my Groq API key is replaced with [REDACTED]. Prompts that mattered most: (1) "add remote and maintain it here ... plan the probe and build it in the most efficient, optimized, production way. i am going away for 1 hour", which turned setup into an autonomous build that measured Groq before designing anything; (2) "when you are finished, reiterate and make it better, repeat till you are satisfied", which produced the measured iterations above (3 s wait cap from a two-seed experiment, lean retrieval, admission-charge estimate, qwen router, 3 s first-token timeout, no retries for best-effort steps), taking overload success from 51.6% (first version, data/experiments/iteration0_standin.csv) to 82.8%; (3) "follow the pdf close ... make no mistakes", after which every claim in this write-up was checked against the data and four of its own draft claims were corrected as unsupported. **What it got wrong:** I did not catch its mistakes myself; it caught them by checking its output against measurements. It first gave the pipeline each simulated person's hidden patience as the deadline, its citation check threw away correct answers because gpt-oss writes "[S1]" with narrow no-break spaces, and its token estimate under-counted Groq's admission charge. **Where I overruled it:** it told me to write the evaluation questions and make the main design calls myself; I chose to have it draft both and then test them with experiments. **What is mine:** the direction and the constraints (production quality, iterate until the measurements stop improving, follow the brief exactly, keep everything in my GitHub repo); the code, the experiments and the first draft of this write-up came from the assistant.